

Programmation Orientée Objet (POO) en Python L2 MPI & SML

Chapitre 3: Encapsulation et Modificateurs d'accès

Pr Amadou Dahirou GUEYE, UAM

Introduction

L'encapsulation est un principe fondamental de la Programmation Orientée Objet (POO) qui consiste à cacher les détails internes d'un objet et à contrôler l'accès à ses attributs et méthodes. Cela permet de :

- Protéger les données contre des modifications involontaires ou incorrectes.

Introduction

- Assurer une meilleure maintenance du code en imposant des règles d'accès.
- Faciliter l'évolution du programme sans risque de casser le fonctionnement existant.

Modificateurs d'accès : Publics, Protégés et Privés

Dans certains langages (comme Java ou C++), il existe des modificateurs `public`, `protected` et `private` pour définir l'accès aux attributs et méthodes d'une classe.

En Python, ces modificateurs ne sont pas strictement appliqués, mais on suit des conventions pour signaler aux développeurs comment utiliser les attributs :

Modificateurs d'accès : Publics, Protégés et Privés

Modificateur	Convention Python	Accès
Public	nom_attribut	Accessible librement partout
Protégé	_nom_attribut	Accessible mais à manipuler avec précaution
Privé	__nom_attribut	Non accessible directement en dehors de la classe

Attributs publics

Un **attribut public** est directement accessible depuis n'importe où.

```
class Voiture:
    def __init__(self, marque, modele):
        self.marque = marque # Attribut public
        self.modele = modele # Attribut public

voiture = Voiture("Toyota", "Corolla")
print(voiture.marque) # OK : Toyota
print(voiture.modele) # OK : Corolla
```

Attributs publics

Problème : On peut modifier ces attributs sans aucune restriction.

Exemple

```
voiture.marque = "Honda" # Modification libre  
print(voiture.marque) # Honda
```

Dans certains cas, on souhaite **restreindre l'accès** pour éviter des modifications involontaires.

Attributs protégés (_nom)

Un attribut protégé commence par un underscore (_). Il est toujours accessible, mais par convention, il ne doit pas être modifié directement.

Attributs protégés (_nom)

Exemple:

```
class Voiture:
    def __init__(self, marque, modele):
        self.marque = marque
        self._modele = modele # Attribut protégé

voiture = Voiture("Toyota", "Corolla")

# Accessible mais déconseillé
print(voiture._modele) # Corolla
voiture._modele = "Camry" # Modification possible mais déconseillée
```

Attributs protégés (_nom)

 **Python ne bloque pas l'accès aux attributs protégés, mais il indique aux développeurs qu'ils ne doivent pas être modifiés directement.**

Attributs privés (__nom)

Un attribut privé commence par un double underscore __. Il n'est pas accessible directement depuis l'extérieur de la classe.

Exemple:

```
class Voiture:
    def __init__(self, marque, modele, prix):
        self.marque = marque
        self._modele = modele
        self.__prix = prix # Attribut privé

voiture = Voiture("Toyota", "Corolla", 20000)

print(voiture.__prix) # ERREUR : AttributeError
```

Attributs privés (__nom)

→ Solution : On peut y accéder en interne via des méthodes dédiées (Getters et Setters).

→ Truc avancé : En interne, Python renomme **__prix** en **_Voiture__prix**, donc on peut y accéder ainsi (mais c'est déconseillé)

:pythonCopierModifier

```
print(voiture._Voiture__prix) # 20000 (Accès possible mais à éviter)
```

Getters et Setters : Contrôler l'accès aux Attributs Privés

Les getters permettent de lire un attribut privé, et les setters permettent de le modifier avec contrôle.

Getters et Setters : Contrôler l'Accès aux Attributs Privés

```
class Voiture:
    def __init__(self, marque, modele, prix):
        self.marque = marque
        self._modele = modele
        self.__prix = prix

    # Getter
    def get_prix(self):
        return self.__prix

    # Setter avec validation
    def set_prix(self, nouveau_prix):
        if nouveau_prix > 0:
            self.__prix = nouveau_prix
        else:
            print("Erreur : Le prix doit être positif.")

voiture = Voiture("Honda", "Civic", 25000)

print(voiture.get_prix()) # 25000
voiture.set_prix(27000) # Modification OK
print(voiture.get_prix()) # 27000
voiture.set_prix(-5000) # Message d'erreur
```

Getters et Setters : Contrôler l'accès aux Attributs Privés

✓ Avantages : Empêche l'accès direct aux attributs privés. Ajoute des règles de validation.

✗ Inconvénient : Oblige à utiliser `get_prix()` et `set_prix()`, ce qui alourdit le code.

→ Solution : **@property** !

Le décorateur @property

Le décorateur `@property` transforme un getter en une propriété qui s'utilise comme un attribut classique.

```
class Voiture:
    def __init__(self, marque, modele, prix):
        self.marque = marque
        self._modele = modele
        self.__prix = prix

    @property
    def prix(self):
        """Getter"""
        return self.__prix
```


Le décorateur @property

```
@prix.setter
def prix(self, nouveau_prix):
    """Setter avec validation"""
    if nouveau_prix > 0:
        self.__prix = nouveau_prix
    else:
        print("Erreur : Le prix doit être positif.")

voiture = Voiture("BMW", "X5", 50000)

print(voiture.prix) # 50000 (utilise le getter)
voiture.prix = 52000 # Appelle automatiquement le setter
print(voiture.prix) # 52000
voiture.prix = -1000 # Message d'erreur
```

Le décorateur @property

✓ Avantages :

- On accède à prix comme un attribut (voiture.prix), mais avec un contrôle interne.
- Plus lisible que get_prix() et set_prix().

Résumé

Concept	Explication
Attribut public	Accessible librement
Attribut protégé (_nom)	Convention : ne pas modifier directement
Attribut privé (__nom)	Non accessible directement
Getter (@property)	Récupère un attribut privé proprement
Setter (@nom.setter)	Modifie un attribut avec validation

Exercice d'application

Crée une classe CompteBancaire avec les attributs suivants :

- titulaire (public)
- solde (privé)

Ajoute :

Exercice d'application

- Un constructeur pour initialiser ces attributs.
- Une méthode `afficher_solde()` pour afficher le solde du compte.
- Un getter `get_solde()` pour récupérer le solde.
- Un setter `set_solde(nouveau_solde)` qui modifie le solde uniquement si `nouveau_solde` est positif.

Conclusion

L'encapsulation est essentielle pour protéger les données et améliorer la maintenabilité du code. L'utilisation de `@property` simplifie l'accès tout en conservant un contrôle strict des modifications.